

Mutable and Immutable

Which values can be edited in place, which cannot, what each method does, and what to check in review

October 2026

The definition. A value is **mutable** if its contents can be edited at its address while labels keep pointing at it. A value is **immutable** if its contents can never be edited; every “change” produces a new value at a new address, and the old one is untouched.

Why it matters.

Mutable values can surprise you through another label (aliasing). Immutable values cannot, but their methods do nothing unless you store the result. Each kind has its own bug.

1. Which is which

Kind	Values	Can be edited in place?
immutable primitives	<code>string</code> , <code>number</code> , <code>boolean</code> , <code>bigint</code> , <code>symbol</code> , <code>null</code> , <code>undefined</code>	never
mutable objects	<code>{}</code> plain objects, <code>[]</code> arrays, <code>Map</code> , <code>Set</code> , <code>Date</code> , class instances, functions, <code>RegExp</code> with state	yes
frozen objects	<code>Object.freeze(o)</code>	no, shallow, silently ignored outside strict mode
<code>readonly</code> typed	<code>readonly number[]</code> , <code>readonly<T></code> , <code>as const</code>	compile-time only; the runtime object is still mutable

Python parallel: `str`, `int`, `float`, `tuple`, `frozenset` immutable; `list`, `dict`, `set` mutable. Same split, same reasons. One difference: Python throws on `s[0] = "x"`; TypeScript silently ignores it.

2. Immutable values: every change is a new value

No string, number or boolean method edits the original. Each returns a new value, and if nothing stores it, the call did nothing.

```
let s = " Ann ";
s.trim();           // builds "Ann"; nobody stores it; s is still " Ann "  ✗ no effect
s = s.trim();       // s now points at "Ann"                               ✓
const t = s.toUpperCase(); // t is "ANN"; s still "Ann"                     ✓ both kept

s[0] = "x";         // silently ignored; s unchanged
s.length = 0;       // silently ignored

let n = 5;
n + 1;              // computes 6; nobody stores it; n is still 5
n = n + 1;          // n is 6
n++;                // n is 7 (shorthand for n = n + 1)
```

Every string method returns new: `trim`, `toUpperCase`, `toLowerCase`, `replace`, `replaceAll`, `slice`, `substring`, `split`, `padStart`, `padEnd`, `concat`, `repeat`, `at`, `charAt`, `normalize`. There is no in-place string method at all.

Consequences.

- The pattern is always *call, then assign*: `x = x.method()` or `const y = x.method()`.
- `x = x.method()` is a reassignment, so `x` must be `let`; under `const`, use a new name.
- Two labels on the same string can never see each other change, so aliasing is impossible for primitives; passing a string to a function cannot change the caller’s string.
- “Build a new one” is how you change a character: `"x" + s.slice(1)`.

Review question.

Any string or number method on its own line with nothing assigned? That line does nothing.

3. Mutable values: some methods edit in place, some return new

Arrays and objects can be edited at their address. Some methods do that; others leave the original alone and return a fresh value. You must know which is which for each call.

Array methods that MUTATE (edit the array in place)

Method	Does	Returns	Trap
<code>push(x)</code>	appends	the new length	<code>const a = arr.push(1)</code> holds a number
<code>pop()</code>	removes last	the removed item	
<code>shift()</code>	removes first	the removed item	O(n)
<code>unshift(x)</code>	prepends	the new length	
<code>splice(i, n, ...items)</code>	removes/inserts at <code>i</code>	array of removed items	
<code>sort(cmp)</code>	reorders	the SAME array	<code>const s = arr.sort()</code> is not a copy
<code>reverse()</code>	reverses	the SAME array	same
<code>fill(v)</code>	overwrites every slot	the SAME array	
<code>copyWithin</code>	copies within	the SAME array	
<code>arr[i] = v</code>	sets a slot	<code>v</code>	
<code>arr.length = 0</code>	empties		all labels see it empty
<code>delete arr[i]</code>	leaves a hole	<code>true</code>	avoid

Array methods that DO NOT mutate (return something new; original untouched)

Method	Returns	Must store?
<code>map(f)</code>	new array, same length	yes
<code>filter(p)</code>	new array, subset	yes
<code>slice(a, b)</code>	new array, copy of a range	yes
<code>concat(b)</code>	new array	yes
<code>flat(), flatMap(f)</code>	new array	yes
<code>toSorted(cmp), toReversed(), toSpliced(), with(i, v)</code>	new array; the non-mutating twins of <code>sort</code> , <code>reverse</code> , <code>splice</code> , <code>arr[i] = v</code>	yes
<code>reduce(f, init)</code>	a value	yes
<code>find(p), findIndex(p), findLast(p)</code>	element / index / <code>undefined</code>	yes
<code>some(p), every(p), includes(x), indexOf(x)</code>	boolean / index	yes
<code>join(sep)</code>	string	yes
<code>at(i)</code>	element	yes
<code>forEach(f)</code>	<code>undefined</code>	no; it is for side effects only
<code>[...arr], Array.from(arr)</code>	new array (shallow copy)	yes

Object operations

Operation	Mutates?
<code>o.k = v, o[k] = v</code>	yes
<code>delete o.k</code>	yes
<code>Object.assign(o, src)</code>	yes, mutates <code>o</code> (the first argument)
<code>Object.assign({}, src)</code>	no; mutates the fresh <code>{}</code>
<code>{ ...o, k: v }</code>	no; new object
<code>Object.keys/values/entries(o)</code>	no; new arrays
<code>Object.freeze(o)</code>	makes <code>o</code> shallowly immutable; returns <code>o</code>
<code>structuredClone(o)</code>	no; deep copy

Map and Set: `set`, `delete`, `clear`, `add` mutate and return the collection (or a boolean); `get`, `has`, `size`, iteration do not.

Date: `setFullYear`, `setMonth`, `setDate`, `setHours`, `setTime` mutate the date in place; `getTime`, `toISOString` do not. `const d = new Date(); d.setDate(1)` edits `d`.

4. The two opposite bugs

Bug A: treating a non-mutating call as if it changed the original.

```
orders.filter((o) => o.paid); // new array built and dropped; orders unchanged
name.trim(); // same
arr.map(x => x * 2); // same
```

Nothing happened. Fix: store the result.

Bug B: treating a mutating call as if it made a copy.

```
const sorted = ids.sort(); // ids is now sorted too; sorted IS ids
const rev = arr.reverse(); // arr is now reversed too
const removed = arr.splice(0, 2); // arr lost two items
```

The original changed, and every label pointing at it sees the change. Fix: copy first (`[...ids].sort()`), or use the non-mutating twin (`ids.toSorted()`).

You see	Ask	If wrong
a method call on its own line	does this method return new?	Bug A: dead line
<code>const x = arr.method()</code>	does this method mutate <code>arr</code> ?	Bug B: <code>x</code> and <code>arr</code> are the same thing, both changed
a method on a parameter	does it mutate?	Bug B on the caller's data
a method on a returned or cached value	does it mutate?	Bug B on shared state

5. `const`, `readonly`, `Object.freeze`: what each actually prevents

Written	Prevents	At	Does not prevent
<code>const a = [1]</code>	<code>a = [2]</code> (re-pointing the label)	compile and runtime	<code>a.push(2)</code> , <code>a[0] = 9</code>
<code>let a: readonly number[]</code>	<code>a.push</code> , <code>a[0] = 9</code> in TS code	compile time only	a JS caller or an <code>as any</code> mutating it
<code>readonly<T></code> , <code>readonly k: T</code>	assignment to those fields	compile time only	mutation through a non-readonly alias of the same object
<code>as const</code>	widening; makes literals <code>readonly</code>	compile time only	runtime mutation
<code>Object.freeze(o)</code>	<code>o.k = v</code> , <code>delete o.k</code> , <code>o.push</code>	runtime, shallow	<code>o.inner.k = v</code> ; silently ignored outside strict mode, throws inside
<code>#field</code>	access from outside the class	runtime	mutation from inside

So `const` is about the label; `readonly` is about the compiler; `Object.freeze` is about the runtime, one level deep. None of them makes a nested object immutable. True deep immutability needs a library or a discipline of never mutating.

6. Copying: how to get a mutable value you can change safely

Written	Copies	Depth
<code>[...a]</code> , <code>a.slice()</code> , <code>Array.from(a)</code>	the array	shallow
<code>{ ...o }</code>	the object	shallow
<code>new Map(m)</code> , <code>new Set(s)</code>	the collection	shallow
<code>a.map(x => ({ ...x }))</code>	the array and one level of its objects	two levels
<code>structuredClone(o)</code>	everything	deep
<code>JSON.parse(JSON.stringify(o))</code>	everything plain; drops <code>undefined</code> , functions, <code>Map</code> , <code>Set</code> ; <code>Date</code> becomes a string	deep-ish

Shallow means the container is new but the things inside are the same objects. `const c = { ...o }`; `c.inner.x = 9` changes `o.inner.x`. `filter` and `map` return shallow copies: the elements are the original objects.

7. Aliasing: the mutable-only hazard

Two labels on one mutable value see each other's edits. This is never a problem for primitives and always a possible problem for objects.

```
const defaults = { retries: 3 };
const opts = defaults;           // alias
opts.retries = 5;                // defaults.retries is 5 for every later caller

const opts2 = { ...defaults };   // copy
opts2.retries = 5;               // defaults untouched
```

Where aliases come from: `b = a`; passing an object to a function; returning an internal array (`return this.items`); storing an object in a cache and also handing it out; `filter/map` results sharing elements with the source; closures capturing an object.

Review question.

For every mutation, name the object and ask who else holds it. For every object handed out of a function or class, ask whether the caller can mutate internal state through it.

8. Immutable style: why some code never mutates

Code that always builds new values instead of editing (`{ ...state, count: state.count + 1 }` rather than `state.count++`) is said to use an immutable style. It is common in React, Redux and functional code. Reading it: every line produces a new value; nothing is edited; `const` everywhere is natural. The one check is Bug A: the new value must be stored or returned.

9. Decision procedure for any line

1. Is the value a primitive (string, number, boolean)? Then nothing can mutate it; the only check is whether the method's result is stored.
2. Is the value an array, object, Map, Set or Date? Find the method or operation in the tables above: mutates, or returns new?
3. If it mutates: who else holds this object? Parameter, shared, returned, cached? That is the finding.
4. If it returns new: is the result assigned, returned or used? If it is alone on a line, that is the finding.
5. If a copy was intended: is it deep enough for what gets edited next?

One sentence to keep.

Immutable: every change is a new value, so store it. Mutable: changes happen in place, so know which methods mutate and who else is holding the object.